

Universitatea Națională de Știință și Tehnologie
POLITEHNICA din București

Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Proiect Microcontrolerul ESP-32
Comunicarea wireless între două plăci ESP32
utilizând Bluetooth Classic și ESP-NOW

Petrescu Daniel-Adrian

I. PREZENTAREA PROIECTULUI:

În cadrul acestui proiect a fost realizată comunicarea wireless între două plăci ESP32 prin intermediul a două tehnologii diferite: **Bluetooth Classic** și **ESP-NOW**. Scopul proiectului este transmiterea stării unui buton de pe o placă ESP32 (Master) către o a doua placă ESP32 (Slave), care controlează aprinderea și stingerea unui LED.

Prin implementarea aceleiași aplicații folosind două metode diferite de comunicație se poate realiza o comparație directă între acestea, evidențiind avantajele și limitările fiecărei soluții. În varianta Bluetooth, comunicația este ușor de implementat și permite conectarea dispozitivelor utilizând mecanismele clasice de asociere specifice protocolului Bluetooth. În schimb, varianta ESP-NOW utilizează un protocol proprietar dezvoltat de Espressif, care oferă o comunicație rapidă, cu latență redusă și consum energetic scăzut, fără a necesita conectarea la o rețea Wi-Fi existentă.

În ambele implementări funcționalitatea este identică: apăsarea butonului pe placa Master determină transmiterea unei comenzi către placa Slave, care aprinde LED-ul, iar eliberarea butonului determină stingerea acestuia. Diferența constă exclusiv în tehnologia de comunicație utilizată pentru transmiterea informației.

II. COMPONENTE NECESARE:

În realizarea proiectului au fost utilizate următoarele componente hardware:

- 2 × ESP32 DevKit
- 1 × Breadboard
- 1 × Buton tactil
- 1 × LED
- 1 × Rezistență de 220 Ω
- Fire de conexiune

Notă: Aceleași componente hardware au fost utilizate atât pentru implementarea comunicației prin Bluetooth, cât și pentru implementarea comunicației prin ESP-NOW. Diferența dintre cele două variante constă exclusiv în protocolul software utilizat pentru transmiterea informațiilor.

III. CONEXIUNILE COMPONENTELOR:

1. ESP32-Master:

Componentă	Pin ESP32
Buton	GPIO 4 (Bluetooth) / GPIO 25 (ESP-NOW)
Celălalt terminal al butonului	GND

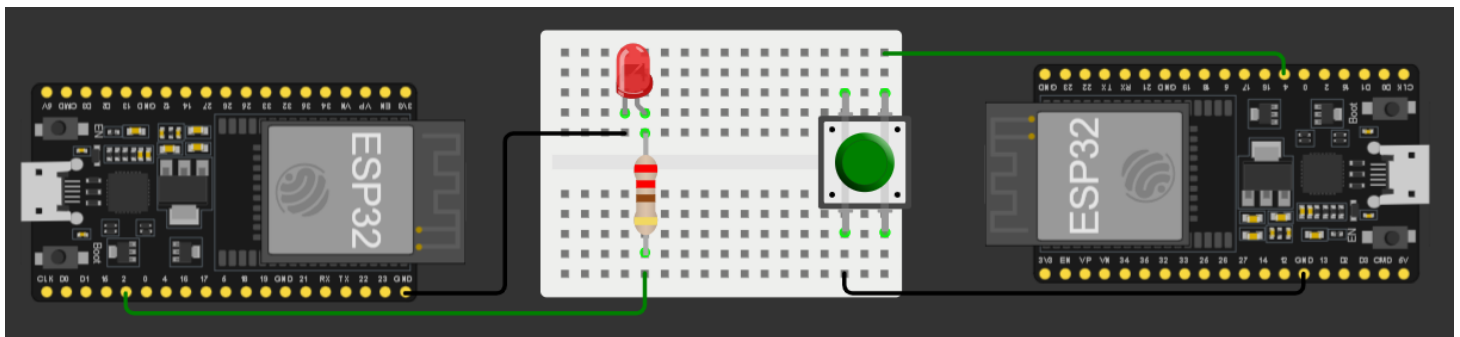
În ambele implementări butonul este configurat utilizând rezistența internă **INPUT_PULLUP**, astfel încât nu este necesară utilizarea unei rezistențe externe. La apăsare, pinul este conectat la masă (GND), iar microcontrolerul detectează nivel logic LOW.

2. ESP32- Slave:

Componentă	Pin ESP32
LED (anod)	GPIO 2
LED (catod)	GND prin rezistență de 220 Ω

LED-ul este comandat de placa Slave și indică starea transmisă de placa Master. Atunci când butonul este apăsat, LED-ul este aprins, iar la eliberarea acestuia LED-ul este stins.

IV. SCHEMA PROIECTULUI:



V. EXPLICATIE PROIECT:

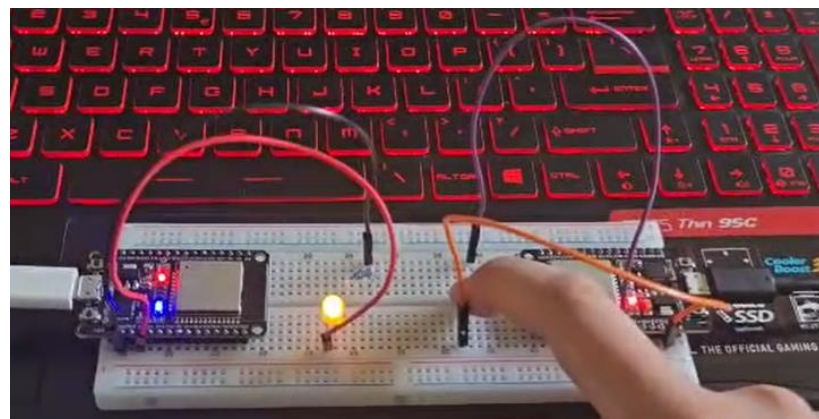
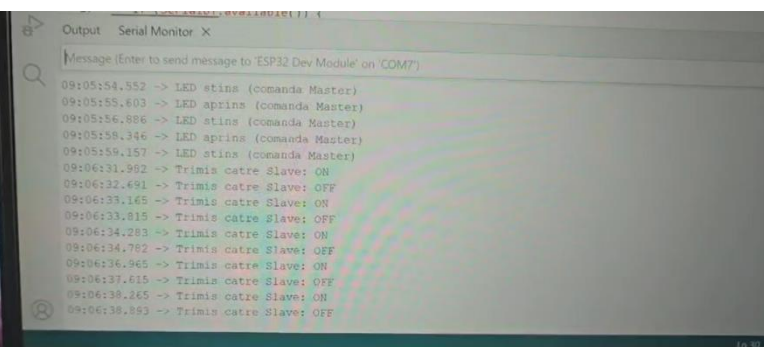
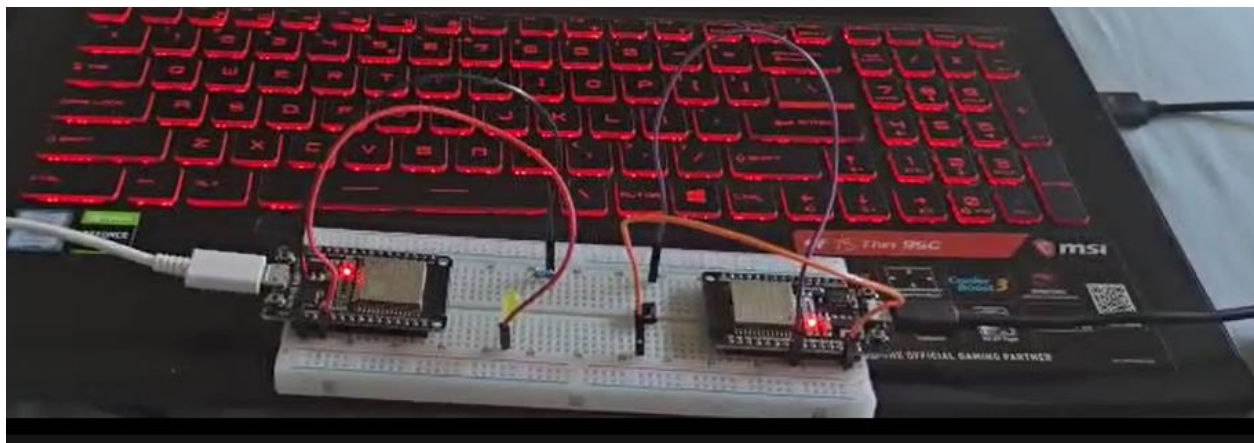
În cadrul acestui proiect au fost implementate două variante ale aceleiași aplicații, utilizând două protocoale diferite de comunicație wireless disponibile pe platforma ESP32: **Bluetooth Classic** și **ESP-NOW**. În ambele cazuri, funcționalitatea este identică: o placă ESP32 configurată ca **Master** citește starea unui buton și transmite această informație către o a doua placă ESP32 configurată ca **Slave**, care controlează aprinderea sau stingerea unui LED.

În prima implementare este utilizat protocolul **Bluetooth Classic**. Placa Master stabilește conexiunea cu placa Slave folosind adresa MAC a acesteia și transmite comenzile „ON” și „OFF” în funcție de starea butonului. La primirea comenzilor, placa Slave interpretează mesajul și modifică starea LED-ului corespunzător.

În cea de-a doua implementare este utilizat protocolul **ESP-NOW**, un protocol proprietar dezvoltat de Espressif pentru comunicații directe între dispozitive ESP32. În acest caz, informația nu mai este transmisă sub formă de text, ci prin intermediul unei structuri de date (struct) care conține starea butonului. Placa Slave primește structura prin intermediul funcției callback și actualizează imediat starea LED-ului.

Realizarea aceleiași aplicații folosind două tehnologii diferite permite compararea acestora din punct de vedere al modului de implementare, al vitezei de transmitere și al complexității configurării. Bluetooth oferă o implementare mai intuitivă și mai ușor de înțeles, în timp ce ESP-NOW asigură o comunicație mai rapidă și un consum redus de energie, fiind recomandat în aplicațiile IoT și în sistemele cu mai multe dispozitive ESP32.

VI. POZE CU PROIECTUL:



VII. COD ARDUINO:

I. COD CONEXIUNI BLUETOOTH:

1. Slave:

```
#include "BluetoothSerial.h"
```

```
BluetoothSerial SerialBT;
```

```
#define LED_PIN 2
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    pinMode(LED_PIN, OUTPUT);
```

```
    digitalWrite(LED_PIN, LOW);
```

```
    SerialBT.begin("ESP32_SLAVE");
```

```
    Serial.println("Slave pornit. Asteptam comenzi...");
```

```
}
```

```
void loop() {
```

```
    if (SerialBT.available()) {
```

```
        String comanda = SerialBT.readStringUntil('\n');
```

```
        comanda.trim();
```

```

if (comanda == "ON") {

    digitalWrite(LED_PIN, HIGH);

    Serial.println("LED aprins (comanda Master)");

} else if (comanda == "OFF") {

    digitalWrite(LED_PIN, LOW);

    Serial.println("LED stins (comanda Master)");

}

}

}

```

2. Master:

```
#include "BluetoothSerial.h"
```

```
BluetoothSerial SerialBT;
```

```
// Adresa MAC a Slave-ului
```

```
uint8_t slaveAddress[] = {0x14, 0x33, 0x5C, 0x38, 0x92, 0xBA};
```

```
#define BUTTON_PIN 4
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
pinMode(BUTTON_PIN, INPUT_PULLUP);
```

```
if (!SerialBT.begin("ESP32_MASTER", true)) {  
    Serial.println("Eroare la pornirea Bluetooth!");  
    while (1);  
}
```

```
Serial.println("Master pornit. Cautam slave...");
```

```
if (SerialBT.connect(slaveAddress)) {  
    Serial.println("Conectat la Slave!");  
} else {  
    Serial.println("Conectare esuata!");  
}  
}
```

```
void loop() {  
    static int lastState = HIGH; // stare buton anterioara  
    int currentState = digitalRead(BUTTON_PIN);  
  
    // detectam apasare  
    if (lastState == HIGH && currentState == LOW) {  
        SerialBT.println("ON");
```

```

    Serial.println("Trimis catre Slave: ON");
}

// detectam eliberare
if (lastState == LOW && currentState == HIGH) {

    SerialBT.println("OFF");
    Serial.println("Trimis catre Slave: OFF");
}

lastState = currentState;

delay(50);
}

```

II. COD CONEXIUNI ESP-NOW:

1. Master:

```

#include <Arduino.h>

#include <esp_now.h>

#include <WiFi.h>

#define BUTTON_PIN 25

// Adresa MAC a Slave-ului (exact cea citită)

```



```
uint8_t slaveAddress[] = {0x14, 0x33, 0x5C, 0x38, 0x92, 0xB8};
```

```
typedef struct struct_message {
```

```
    bool buttonState;
```

```
} struct_message;
```

```
struct_message myData;
```

```
esp_now_peer_info_t peerInfo;
```

```
// Detectăm IDF v5 (folosește wifi_tx_info_t)
```

```
#ifdef ESP_NOW_MAX_TOTAL_PEER_NUM
```

```
    #define USE_IDF_V5
```

```
#endif
```

```
// Callback trimitere
```

```
#ifdef USE_IDF_V5
```

```
void OnDataSent(const wifi_tx_info_t *info, esp_now_send_status_t status) {
```

```
    Serial.print("Status trimitere către: ");
```

```
    if (info) {
```

```
        for (int i = 0; i < 6; i++) {
```

```
            Serial.printf("%02X", info->des_addr[i]);
```

```
            if (i < 5) Serial.print(":");
```

```
        }
```

```

}

Serial.print(" -> ");

Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Succes" : "Eşec");

}

#else

void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {

    Serial.print("Status trimitere către: ");

    for (int i = 0; i < 6; i++) {

        Serial.printf("%02X", mac_addr[i]);

        if (i < 5) Serial.print(":");

    }

    Serial.print(" -> ");

    Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Succes" : "Eşec");

}

#endif

void setup() {

    Serial.begin(115200);

    pinMode(BUTTON_PIN, INPUT_PULLUP);

    WiFi.mode(WIFI_STA);

    if (esp_now_init() != ESP_OK) {

```

```
Serial.println("Eroare la init ESP-NOW!");  
  
return;  
  
}
```

```
esp_now_register_send_cb(OnDataSent);
```

```
memcpy(peerInfo.peer_addr, slaveAddress, 6);
```

```
peerInfo.channel = 0;
```

```
peerInfo.encrypt = false;
```

```
if (esp_now_add_peer(&peerInfo) != ESP_OK) {  
    Serial.println("Eroare la adaugarea slave-ului!");  
    return;  
}
```

```
Serial.println("Master pornit!");  
  
}
```

```
void loop() {
```

```
    myData.buttonState = (digitalRead(BUTTON_PIN) == LOW);
```

```
    esp_err_t result = esp_now_send(slaveAddress, (uint8_t *) &myData, sizeof(myData));
```

```
    if (result == ESP_OK) {
```

```
    Serial.println("Mesaj trimis");

} else {

    Serial.println("Eroare la trimitere!");

}

delay(500);

}
```

2. Slave:

```
#include <Arduino.h>

#include <esp_now.h>

#include <WiFi.h>

#define LED_PIN 2

typedef struct struct_message {

    bool buttonState;

} struct_message;

struct_message myData;

#ifdef ESP_NOW_MAX_TOTAL_PEER_NUM
```

```

void OnDataRecv(const esp_now_recv_info_t *info, const uint8_t *incomingData, int len) {

    memcpy(&myData, incomingData, sizeof(myData));

    Serial.print("Date primite de la: ");

    for (int i = 0; i < 6; i++) {

        Serial.printf("%02X", info->src_addr[i]);

        if (i < 5) Serial.print(":");

    }

    Serial.print(" -> Buton: ");

    Serial.println(myData.buttonState ? "APĂSAT" : "NEAPĂSAT");

    digitalWrite(LED_PIN, myData.buttonState ? HIGH : LOW);

}

```

#else

```

void OnDataRecv(const uint8_t *mac, const uint8_t *incomingData, int len) {

    memcpy(&myData, incomingData, sizeof(myData));

    Serial.print("Date primite de la: ");

    for (int i = 0; i < 6; i++) {

        Serial.printf("%02X", mac[i]);

        if (i < 5) Serial.print(":");

    }

    Serial.print(" -> Buton: ");

    Serial.println(myData.buttonState ? "APĂSAT" : "NEAPĂSAT");

}

```

```
    digitalWrite(LED_PIN, myData.buttonState ? HIGH : LOW);  
}
```

```
#endif
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    pinMode(LED_PIN, OUTPUT);
```

```
    WiFi.mode(WIFI_STA);
```

```
    if (esp_now_init() != ESP_OK) {
```

```
        Serial.println("Eroare la init ESP-NOW!");
```

```
        return;
```

```
    }
```

```
    esp_now_register_recv_cb(OnDataRecv);
```

```
    Serial.println("Slave pornit!");
```

```
}
```

```
void loop() {
```

```
    // Nimic aici, callback-ul face totul
```

```
}
```

SIMULARE: <https://wokwi.com/projects/469883589843055617>

VIII. CONCLUZII:

Prin realizarea acestui proiect au fost studiate și comparate două dintre cele mai utilizate metode de comunicație wireless disponibile pentru platforma ESP32: **Bluetooth Classic** și **ESP-NOW**. Deși ambele variante îndeplinesc aceeași funcționalitate, modul de implementare și caracteristicile fiecărui protocol sunt diferite.

Implementarea prin Bluetooth este mai intuitivă și mai ușor de configurat, fiind potrivită pentru aplicații simple în care dispozitivele trebuie împerecheate și controlate direct. În schimb, ESP-NOW oferă o comunicație rapidă, cu latență redusă și consum energetic scăzut, fiind recomandat pentru aplicații IoT și sisteme distribuite formate din mai multe dispozitive ESP32.

Realizarea aceleiași aplicații folosind două protocoale diferite a permis înțelegerea avantajelor și limitărilor fiecărei tehnologii și a demonstrat flexibilitatea platformei ESP32 în dezvoltarea aplicațiilor de comunicație wireless.

IX. REFERINTE:

- Arduino, **ESP32 Arduino Core Documentation**.
- Espressif Systems, **ESP-NOW API Reference**.
- Arduino, **BluetoothSerial Library Documentation**.
- Arduino IDE Documentation.
- ESP32 DevKit V1 – Technical Specifications.
- Echipa **SUMMER IT 2025 – ETTI**.